

Eco-Matic Vending Machine

1. Project Overview

Purpose

Eco-Matic is a console-based vending machine simulator written in C#. It simulates customer and administrative interactions while promoting sustainability by letting users recycle materials for machine credit. The application is data-driven—inventory and event logs are persisted to CSV files so the machine state can be inspected or restored between runs.

Features

- Customer-facing purchasing flow (browse, insert money, buy, examine items).
- Recycle-for-credit functionality (plastic, glass, aluminum) that adds to customer balance.
- Admin panel (password-protected) for viewing logs, restocking, adding/removing items, and generating daily sales reports.
- CSV-backed dynamic inventory with automatic validation and repair.
- Colorized console UI using Spectre.Console for tables, prompts, and status messages.
- Transaction receipts consolidating identical purchases and recycle entries.

Target Audience

- Instructors and students learning C# and console I/O.
- Developers prototyping vending-machine logic or sustainability incentives.
- Demonstrations or class projects that showcase OOP, file handling, and basic UX in a terminal.

Technology Stack

- Language: C# (targets .NET 9 runtime).
- Libraries: Spectre.Console (for colorized output and tables).
- File formats: CSV (`inventory.csv`, `eventLog.csv`) placed under a `data/` folder.

2. Requirements

Software Requirements

- .NET 9 SDK (or compatible .NET 9 runtime).
- An IDE such as Visual Studio 2022 or Visual Studio Code.

Installation & Run Steps

1. Clone repository and open terminal in repository root.
2. Restore and build dependencies:

```
dotnet restore  
dotnet build eco-matic/eco-matic.csproj
```

3. Run the program:

```
dotnet run --project eco-matic/eco-matic.csproj
```

The program automatically creates a `data` directory and the default `inventory.csv` and `eventLog.csv` files if missing.

System/Hardware Requirements

- Any modern PC capable of running the .NET 9 runtime.

3. File Handling Overview

Files & Purpose

- `data/inventory.csv`: Source of truth for the machine inventory. Columns: `Type, Name, Price, Stock, Calories/Volume`.
- `data/eventLog.csv`: Transaction and admin event log. Header: `Timestamp, EventType, Action, ItemName, UnitPrice, Quantity, TotalPrice, Details`.

File Operations Used

- Read: Loading inventory (`LoadInventory`) and reading event log (`ViewEventLog, SalesReport`).
- Write (overwrite): Updating inventory when item states change (`UpdateInventory, CreateDefaultInventoryFile`).
- Append: Appending event lines to `eventLog.csv` (`LogEvent`).
- Create: Creating `data` directory and default CSV files (`CheckDirectory, CreateDefaultInventoryFile, CreateDefaultEventLogFile`).

Error Handling

- The application validates CSV headers and row formats (`CheckInventoryFile, CheckEventLogFile`) and automatically recreates the default files if irregularities are discovered.
- File operations are wrapped with try/catch blocks to provide user-friendly `Write.Error` outputs if `IOExceptions` occur.

4. Code Structure

Main Program Structure

All code is currently in a single `Program.cs` file. The major classes and utilities are:

- `Program` — Entry point and interactive menu logic for Customer and Admin modes.
- `EcoMatic` — Core machine class: inventory state, balance, recycling settings, accepted bills, and the principal business logic (`InsertMoney`, `BuyItem`, `RecycleForCredit`, `GetChange`, `Restock`, `AddItem`, `RemoveItem`, `ShowInventory`, file checks, and logging).
- `VendingItem` (abstract) — Base class for items with child classes: `SnackItem` (`IHasCalories`), `DrinkItem` (`IHasVolume`), `MiscItem`. Polymorphic methods supply dispense and examine messages and CSV serialization.
- `TransactionTracker` & `RecycleTracker` — Simple array-backed trackers for the current session's purchases and recycled items (aggregation logic included).
- `ReceiptPrinter` — Prints consolidated session receipt to the console.
- `SalesReport` — Parses `eventLog.csv` and prints a daily sales report.
- `Write` — Small utility with `DelayLine`, `DelayLoad`, and formatted `Warning/Error/Success` outputs.

Key constants & fields (Can be changed according to user)

- `EcoMatic.MaxItems = 15` — maximum item slots.
- `EcoMatic.MaxStocks = 10` — maximum stock per item.
- `EcoMatic.AcceptedBills` — accepted denominations: 20, 50, 100, 200, 500, 1000.
- `EcoMatic.RecyclableItems` and `RecycleItemsPricePerGram` — configured recyclable types and prices.

Code Walkthrough — key methods

- `EcoMatic.InsertMoney(decimal amount)` — Validates bill denominations and increases `CurrentBalance`. Logs the insertion.
- `EcoMatic.BuyItem(int id)` — Validates id, stock, and balance; dispenses item (polymorphic message), decrements stock, updates CSV, logs the purchase, and records it in `TransactionTracker`.
- `EcoMatic.RecycleForCredit(int id, double grams)` — Validates recyclable choice and grams range (1–5000), computes credit, increments `CurrentBalance`, logs and records the recycle entry.
- `EcoMatic.GetChange()` — Prints receipt with `ReceiptPrinter`, returns change if any, logs the return, and clears session trackers.
- `EcoMatic.Restock(int id)` — Fills an item stock up to `MaxStocks`, updates CSV, and logs the restock.

- Inventory file routines: `CheckInventoryFile()`, `CreateDefaultInventoryFile()`, `LoadInventory()`, and `UpdateInventory()` manage validation, default creation, loading to runtime arrays, and saving changes.

Modularity & Reusability

- OOP is used to separate item behavior via inheritance and interfaces. However, the code is monolithic within one file; moving classes to separate files would improve maintainability.
- Trackers and the receipt printer are reusable components but are implemented with fixed-size arrays rather than collections.

5. User Interface

Design & Usability

- Menu-driven console UI with clear separation between `Customer` and `Admin` flows.
- `Spectre.Console` provides colored tables, prompts, and status messages, improving readability and highlighting important information (balance, stock statuses, warnings).

Inputs & Outputs

- Inputs: textual selection prompts, numeric inputs (IDs, bill amounts, weights), and password input for Admin (masked).
- Outputs: Inventory table (ID, Item, Price, Stock, Status), colorful success/error/warning lines, receipts, and sales reports.

Example flows & sample interactions

- Insert Money: user types an accepted bill (e.g., `100`). The app validates and updates the balance.
- Buy Item: user provides item ID (e.g., `3`). If balance is sufficient and stock available the item is dispensed and stock reduced.
- Recycle: choose item number (1–3 for plastic/glass/aluminum), provide weight in grams, receive credit added to balance.

Error Messages

- User-friendly error/warning messages use the `Write.Error`, `Write.Warning`, and `Write.Success` helpers. Examples include `Invalid bill amount.`, `Invalid Range.`, `Out of stock.`, and `Invalid weight. Please enter a value between 1 and 5000 grams.`

6. Challenges and Solutions

Challenges encountered

- Ensuring CSV integrity when users may accidentally edit data or when files are missing.
- Aggregating session transactions for clean receipts without using dynamic collections.
- Keeping a usable UI/UX within a purely console environment.

Solutions & Workarounds

- Implemented `CheckInventoryFile()` and `CheckEventLogFile()` that validate file headers/rows and recreate default files when invalid.
- Implemented aggregation logic in `TransactionTracker` and `RecycleTracker` to merge identical items using array-search rather than dynamic lists.
- Used Spectre.Console to create readable tables and colored status indicators to compensate for lack of GUI.

7. Testing

Test cases (representative)

- Valid purchase: insert an accepted bill (e.g., `100`) and buy an item priced `30.50`, stock decremented, receipt shows purchase.
- Insufficient funds: attempt to buy an item with less than item price, shows an error and does not decrement stock.
- Out of stock: attempt to buy an item with stock `0`, returns `out of stock` message.
- Invalid bill: insert an unsupported bill amount, shows `Invalid bill amount`.
- Recycle validations: weight below `1` or above `5000` shows error; valid weight credits balance correctly.
- Admin password: wrong password denies access.
- File integrity: remove or corrupt `inventory.csv` header, program reconstructs default file automatically.

Results & Limitations

- The program handles the typical flows satisfactorily and recovers from missing or corrupt CSV files by recreating defaults.
- Limitations include fixed-size arrays for inventory and trackers (not scalable beyond configured limits) and the monolithic single-file layout.

8. Future Enhancements

Planned features

- Replace fixed arrays with dynamic collections (`List<T>`, `Dictionary<T>`) for scalability and easier code.
- Split classes across multiple files and add namespaces for clarity and compile-time organization.
- Use JSON for all stored data (inventory, logs, and recyclable settings), and have the program read and save information through JSON files instead of CSV.
- Add multi-day / historical sales reporting, inventory valuation, and restock history.

Performance & UI/UX Improvements

- Make the program faster by loading data once and saving only when there are changes.
- Graphical User Interface (GUI), such as using Windows Forms or WPF, instead of a console view.

9. Conclusion

Reflection

This project demonstrates practical C# skills: object-oriented design, file I/O, input validation, and crafting a user-friendly console application with a sustainability goal. The constraints (array-based trackers, single-file layout) were deliberate choices for clarity since I still haven't fully understood DSA (still self-learning) but highlight clear next steps for production readiness.

Takeaways

- Solid grasp of file handling patterns in C#, basic logging, and simple report generation.
- Hands-on experience integrating a third-party console UI library (Spectre.Console) to improve terminal usability.

Appendix

Source Files

- Github – <https://github.com/SeanixReal/Eco-Matic/>

References

- Spectre.Console – <https://spectreconsole.net/>
- Dotnet Documentation – <https://learn.microsoft.com/en-us/dotnet/>